



Security Review For Mezzanine



Collaborative Audit Prepared For:
Lead Security Expert(s):

Mezzanine
0x73696d616f
TIMOH

Date Audited:

February 26 - March 5, 2026

Introduction

Mezzanine is a structured yield protocol built around a three-tranche system. Users deposit USDC to mint mzUSD, which can be held as a liquid base asset or staked into one of three yield-bearing tranches: Senior (smzUSD), Mezzanine (MLT), and Junior (JLT).

Each tranche represents a defined position in the loss waterfall. JLT absorbs losses first and receives the highest risk premium. MLT sits in the second loss position. smzUSD absorbs losses last and earns base yield with no risk premium. Capital is deployed into whitelisted DeFi strategies via institutional MPC custody, with all strategy wallets publicly visible on-chain.

Scope

Repository: [woof-software/mezzanine-sc](https://github.com/woof-software/mezzanine-sc)

Audited Commit: [4b022b77389620e057abd9ea49b5ff86c49b482b](#)

Final Commit: [7c05a6c328462ddd7cd44db2814267350e6438f2](#)

Files:

- [contracts/AccessManagerImport.sol](#)
- [contracts/automation/NavOracleAutomation.sol](#)
- [contracts/automation/UmbrellaEpochFinalizer.sol](#)
- [contracts/interfaces/ILossManager.sol](#)
- [contracts/interfaces/IMZUSD.sol](#)
- [contracts/interfaces/INavOracleAutomation.sol](#)
- [contracts/interfaces/IReservoir.sol](#)
- [contracts/interfaces/ITranche.sol](#)
- [contracts/interfaces/IVault.sol](#)
- [contracts/IssuanceVault.sol](#)
- [contracts/LossManager.sol](#)
- [contracts/MZUSD.sol](#)
- [contracts/pause/IPauseManager.sol](#)
- [contracts/pause/PauseCategory.sol](#)
- [contracts/pause/PauseManagedUpgradeable.sol](#)
- [contracts/pause/PauseManager.sol](#)
- [contracts/PolJuniorTranche.sol](#)
- [contracts/ReservoirUSDC.sol](#)

- `contracts/Tranche.sol`

Final Commit Hash

7c05a6c328462ddd7cd44db2814267350e6438f2

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	2	7

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Current oracle design is prone to errors

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/27>

Summary

NAV and Supply oracles will return answers in different time points. It means they can't be compared to calculate deficit/profit because belong to different time points.

Vulnerability Detail

NavOracleAutomation.sol uses 2 custom Chainlink oracles to fetch NAV and MZUSD supply:

```
function _readFreshAnswer(
    AggregatorV3Interface navOracle_,
    AggregatorV3Interface totalSupplyOracle_
) internal view returns (uint256 navUSDC, uint256 totalSupplyMZ) {
    ...

    (, int256 navAnswer, , uint256 navUpdatedAt, ) = navOracle_.latestRoundData();
    ...
    navUSDC = uint256(navAnswer);

    (, int256 supplyAnswer, , uint256 supplyUpdatedAt, ) =
    ↪ totalSupplyOracle_.latestRoundData();
    ...
    totalSupplyMZ = uint256(supplyAnswer);
}
```

Then it compares values, if $NAV > supply$ it distributes yield, otherwise distributes loss.

Problem is that those are 2 distinct oracles. Oracles can be updated in different timestamps and therefore will reflect different state. Suppose following example:

- 1) At timestamp 1000 there are 100 NAV and 100 supply.
- 2) Supply oracle is updated with `answer = 100`.
- 3) At timestamp 1010 user deposits 500 USDC to MZUSD.sol.
- 4) NAV oracle is updated with `answer = 600` - which is correct for the current timestamp.
- 5) Protocol will compare those 2 values and calculate yield $600 - 100 = 500$ to distribute. Note both 2 values are correct, but they belong to different timestamps so can't be compared.

Impact

System will calculate incorrect amount of NAV profit/deficit, which means core protocol mechanism is broken.

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/automation/NavOracleAutomation.sol#L230-L266>

Tool Used

Manual Review

Recommendation

Both NAV and Supply oracles are custom, so maybe implement single custom oracle which already returns `deltaNav`. I.e. Chainlink will compare NAV and MZUSD supply at certain point in time and submit this as answer.

Issue M-1: LossManager applies loss to disabled tranches

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/24>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Disabled tranches don't receive yield, however they do share loss which is incorrect.

Vulnerability Detail

In yield distribution it skips not enabled tranches:

```
for (uint256 i; i < trancheCount; ++i) {
    t = tranches[i];
    trancheTvl = ITranche(t.tranche).totalAssets();
    if (!t.enabled || trancheTvl == 0) continue;
    trancheTvls[i] = trancheTvl;
    totalTvl += trancheTvl;
    // Track the last tranche that actually participates in allocation.
    // Dust will be assigned there to avoid leftover MZUSD on the IssuanceVault.
    lastNonZeroIndex = i;
}
```

However there is no such check in loss distribution, it distributes between all tranches. So users will lose money even when tranche is discontinued.

Impact

LossManager applies loss to disabled tranches. There is no way to remove tranche, so discontinued tranches will be marked as disabled and will share loss forever.

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/LossManager.sol#L154-L199>

Tool Used

Manual Review

Recommendation

Skip disabled tranches in `LossManager._applyLossAcrossOrder()`

Issue M-2: NavOracleAutomation.performUpkeep() can revert wasting funds

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/26>

Summary

Chainlink Automation calls `checkUpkeep()` offchain and executes `performUpkeep()` when it returns true. However it doesn't check some cases which cause revert. As a result, Chainlink will submit reverting transactions which consume funds.

Vulnerability Detail

`checkUpkeep()` returns true as soon as `vetoPeriod` has passed.

```
function checkUpkeep(
    bytes calldata checkData
) external view returns (bool upkeepNeeded, bytes memory performData) {
    checkData;
    if (!hasPendingNav()) return (false, "");
    @> if (block.timestamp < pendingReadyAt) return (false, "");
    return (true, "");
}

function hasPendingNav() public view returns (bool) {
    return pendingReadyAt != 0;
}

function performUpkeep(bytes calldata performData) external {
    performData;
    if (!hasPendingNav()) revert NoPendingNav();
    uint256 readyAt = pendingReadyAt;
    if (block.timestamp < readyAt) revert NotReady(readyAt);
    _applyPending();
}

function _applyPending() internal {
    bool isYield = pendingIsYield;
    uint256 deltaUSDC = pendingDeltaUSDC;
    uint256 readyAt = pendingReadyAt;

    _clearPending();

    if (isYield) {
        IVault(vault).distributeYield(deltaUSDC);
    } else {
```

```

        ILossManager(lossManager).applyLoss(deltaUSDC);
    }
}

```

However there are other conditions when `IssuanceVault.distributeYield()` and `LossManager.applyLoss()` can revert:

- 1) Both execute `IssuanceVault._advanceEpochAndValidate()` which ensures enough time has passed in current epoch. So it will revert if `epochDuration > vetoPeriod`.

```

function _advanceEpochAndValidate() internal returns (uint256 newEpochId) {
    uint64 ts = uint64(block.timestamp);
@>    if (ts < epochStart + epochDuration) revert EpochTooSoon();
    ...
}

```

- 2) It reverts if loss amount is too big:

```

function applyLoss(
    uint256 usdcLoss
) external onlyNavOracle whenNotPaused(PauseCategory.PROTOCOL) {
    if (usdcLoss == 0) revert ZeroAmount();
@>    if (maxLossPerTxUSDC != 0 && usdcLoss > maxLossPerTxUSDC) revert
↪    LimitExceeded();
}

```

- 3) Reverts when there are no tranches:

```

function distributeYield(
    uint256 usdcReported
) external nonReentrant onlyNavOracle whenNotPaused(PauseCategory.PROTOCOL) {
    if (tranches.length == 0) revert NoTranchesAdded();
}

```

Impact

Chainlink Automation can execute reverting transactions which consumes protocol funds. Mainly when `epochDuration > vetoPeriod`. It is reasonable to assume because `MAX_EPOCH_DURATION_LIMIT = 30 days`, while `MAX_VETO_PERIOD = 5 days`.

However it's not clear what is Chainlink's behaviour when `performUpkeep()` reverts multiple times in a row.

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/automation/NavOracleAutomation.sol#L157-L175>

Tool Used

Manual Review

Recommendation

As a best practice `checkUpkeep()` should perform all those checks implemented in called functions.

Issue L-1: Tranche is not compatible with EIP4626

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/20>

Summary

Functions `maxMint()` and `maxDeposit()` by default return `2256-1` and are not overridden in `Tranche.sol`. However actually there is deposit limit.

Vulnerability Detail

Here you can see it includes limit check during deposit:

```
/// @notice Deposit assets into the tranche, respecting deposit limits.
/// @dev Overrides ERC4626 _deposit to enforce deposit limits and pause.
function _deposit(
    address caller,
    address receiver,
    uint256 assets,
    uint256 shares
) internal override whenNotPaused(PauseCategory.DEPOSITS) {
    _checkDepositLimit(assets);
    super._deposit(caller, receiver, assets, shares);
}

/// @dev Deposit limits apply only to user deposits/mints; yield streaming via
    ↪ notifyYield
/// does not check these limits and can continue even when deposits would revert.
// Global deposit limits (cap + % of total supply).
function _checkDepositLimit(uint256 amount) internal view {
    (uint256 limit, , ) = _computeDepositLimit();
    if (amount > limit) revert DepositLimitExceeded(amount, limit);
}
```

Impact

Functions `maxMint()` and `maxDeposit()` return incorrect values breaking ERC4626 compatibility.

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/Tranche.sol#L552-L555>

Tool Used

Manual Review

Recommendation

Override `maxMint()` and `maxDeposit()` to include deposit cap logic.

Issue L-2: TRANCHE_WEIGHT_LIMIT is not used

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/21>

Description

Contract declares constant, but never uses it.

Vulnerability Detail

IssuanceVault has constant TRANCHE_WEIGHT_LIMIT which is never used.

Impact

No impact

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/IssuanceVault.sol#L42>

Tool Used

Manual Review

Recommendation

Consider removing TRANCHE_WEIGHT_LIMIT.

Issue L-3: Incorrect slot packing in NavOracleAutomation

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/22>

Summary

Comment incorrectly assumes variables are packed into 1 slot.

Vulnerability Detail

It assumes all variables are packed into slot 0, however it sums to 33 bytes. So it means that uint32 maxAnswerStaleness takes slot 1:

```
// Packed storage:
// slot 0: pendingDeltaUSDC (16 bytes) + pendingReadyAt (8 bytes) + pendingIsYield
//           ↳ (1 byte)
//           + vetoPeriod (4 bytes) + maxAnswerStaleness (4 bytes)
// uint128 comfortably covers practical USDC (6d) deltas with wide headroom.
uint128 public pendingDeltaUSDC;
// uint64 timestamp and uint32 durations match protocol style (IssuanceVault epoch
//           ↳ config) and are ample.
uint64 public pendingReadyAt;
// Direction is stored separately to avoid signed-int casts/abs math during upkeep
//           ↳ execution.
bool public pendingIsYield;
uint32 public vetoPeriod;
uint32 public maxAnswerStaleness;
```

Impact

Confusing comment

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/automation/NavOracleAutomation.sol#L29-L39>

Tool Used

Manual Review

Recommendation

Consider reducing size of variables to pack it into 1 slot.

Issue L-4: Incorrect condition in `Tranche.requestWithdrawal()`

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/23>

Summary

Current logic overrides snapshot price in case it's 0 instead of calculating average price. It can be used to reduce fee on positive price movement during withdrawal.

Vulnerability Detail

`Tranche.requestWithdrawal()` saves `userSnapshotPrice` to then take fee in case during withdrawal delay price rises. As you can see, it overrides `userSnapshotPrice` in case user submitted request already, but `userSnapshotPrice == 0`:

```
function requestWithdrawal(uint256 shares) external
↳ whenNotPaused(PauseCategory.WITHDRAWALS) {
    ...

    uint256 currentVaultEpoch = IVault(vault).currentEpochId();
    uint256 targetEpoch = currentVaultEpoch + unstakeDelayEpochs;

    uint256 existingShares = userSharesByEpoch[msg.sender][targetEpoch];
    uint256 currentPrice = convertToAssets(10 ** decimals());
    // For repeat requests in the same target epoch keep weighted-average
    ↳ snapshot.
    // If snapshot is not initialized (possible after upgrade for old pending
    ↳ requests), initialize to current.
@> if (existingShares == 0 || userSnapshotPrice[msg.sender][targetEpoch] == 0)
↳ {
@>     userSnapshotPrice[msg.sender][targetEpoch] = currentPrice;
} else {
    uint256 oldSnapshot = userSnapshotPrice[msg.sender][targetEpoch];
    uint256 totalUserShares = existingShares + shares;
    userSnapshotPrice[msg.sender][targetEpoch] =
        ((oldSnapshot * existingShares) + (currentPrice * shares)) /
        ↳ totalUserShares;
}
    ...
}
```

Such behaviour is incorrect, because there is legitimate case when `existingShares != 0` and price is indeed 0. Consider following example:

- 1) Suppose `shares = 1000e21`, `totalAssets = 0` - that's because full burn was

executed by LossManager in epoch N.

- 2) There is yield = 500e18 in epoch N+1. User calls requestWithdraw(1000e21 - 1) in the same block after notifyYield() is called. It writes userSharesByEpoch = 1000e21 - 1 and snapshotPrice = 0.
- 3) In the last block of epoch N+1 user calls requestWithdraw(1 wei). Due to bug it sets current price to snapshotPrice, which is 0.5e18.
- 4) Assume no more yield and loss until epoch finalisation for simplicity.
- 5) As a result, it won't calculate fee because snapshot is equal to finalisation price. However in reality average price is 0, so fee must take place on 0 -> 0.5e18 movement.

Impact

User is able to pay less fee on withdrawal in certain edge case.

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/Tranche.sol#L196>

Tool Used

Manual Review

Recommendation

Remove this condition:

```
-      if (existingShares == 0 || userSnapshotPrice[msg.sender][targetEpoch] == 0)
↪    {
+      if (existingShares == 0) {
          userSnapshotPrice[msg.sender][targetEpoch] = currentPrice;
      } else {
          uint256 oldSnapshot = userSnapshotPrice[msg.sender][targetEpoch];
          uint256 totalUserShares = existingShares + shares;
          userSnapshotPrice[msg.sender][targetEpoch] =
              ((oldSnapshot * existingShares) + (currentPrice * shares)) /
              ↪ totalUserShares;
      }
```

Issue L-5: Last tranche receives risk premium of disabled tranches

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/25>

Summary

There is possible state when tranche is not enabled, but has positive `riskPremiumBps`. Such state is incorrectly handled in yield distribution.

Vulnerability Detail

`IssuanceVault.setTrancheConfig()` ensures that `riskPremiumBps` sums to `BPS = 10_000`. Worth noting that it never checks that configured tranches are all enabled.

```
function setTrancheConfig(
    TrancheInfo[] calldata _tranches,
    bool[] calldata _isPairedWithNext,
    uint16 _baseRewardBps
) external restricted whenNotPaused(PauseCategory.ADMIN) {
    ...

    // Validate registered, no duplicates, and sum yield to BPS.
    uint256 total = _baseRewardBps;
    for (uint256 i; i < len; ++i) {
        address tranche = _tranches[i].tranche;
        if (!isTranche[tranche]) revert TrancheNotRegistered(tranche);
        for (uint256 j; j < i; ++j) {
            if (_tranches[j].tranche == tranche) revert
                ↪ DuplicateTranche(tranche);
        }
        @> total += _tranches[i].riskPremiumBps;
    }
    @> if (total != BPS) revert InvalidYieldDistribution();
    ...
    // Overwrite tranches array, validate pair contiguity and build pair
    ↪ mapping.
    @> tranches = _tranches;
    ...
}
```

In case there is not enabled tranche with positive `riskPremiumBps`, it will be skipped in yield distribution. However their share is treated as remainder and will be distributed to last tranche:

```
function _allocateYieldToTranches(
```

```

        uint256 allocatableMZUSD
    ) internal returns (uint256 actuallyAllocated) {
        ...
        for (uint256 i; i < trancheCount; ++i) {
            t = tranches[i];
            trancheTvl = ITranche(t.tranche).totalAssets();
@>            if (!t.enabled || trancheTvl == 0) continue;
            trancheTvls[i] = trancheTvl;
            totalTvl += trancheTvl;
            // Track the last tranche that actually participates in allocation.
            // Dust will be assigned there to avoid leftover MZUSD on the
            ↪ IssuanceVault.
            lastNonZeroIndex = i;
        }
        ...
        for (uint256 i; i < trancheCount; ) {
            ...
            uint256 pairTotal;
@>            if (i == lastNonZeroIndex || i + 1 == lastNonZeroIndex) {
@>                pairTotal = allocatableMZUSD - allocated;
            } else {...}
            ...
        }
        ...
        uint256 totalShare;
@>        if (i == lastNonZeroIndex) {
@>            totalShare = allocatableMZUSD - allocated;
        } else {...}
        ...
    }
    ...
}

```

Impact

Last tranche receives risk premium of disabled tranches, basically stealing from other tranches.

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/IssuanceVault.sol#L775-L776>
<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/IssuanceVault.sol#L735-L736>

Tool Used

Manual Review

Recommendation

Ensure `riskPremiumBps == 0` when `enabled = false` in `IssuanceVault.setTrancheConfig()`

Issue L-6: MAX_TRANCHES can be bypassed

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/28>

Summary

Protocol incorrectly enforces max number of tranches.

Vulnerability Detail

Config consists of 2 parts:

- 1) registerTranche() - marking isTranche[_tranche] = true. Here it performs max length check

```
function registerTranche(
    address _tranche
) external restricted whenNotPaused(PauseCategory.ADMIN) {
    if (_tranche == address(0)) revert ZeroAddress();
    if (isTranche[_tranche]) revert TrancheExists();
    @> if (tranches.length >= MAX_TRANCHES) revert TooManyTranches();

    @> tranches.push(TrancheInfo(_tranche, 0, false));
    isTranche[_tranche] = true;
    emit TrancheRegistered(_tranche);
}
```

- 2) setTrancheConfig() - real config. Here it checks that tranche is registered and then overwrites tranches array:

```
function setTrancheConfig(
    TrancheInfo[] calldata _tranches,
    bool[] calldata _isPairedWithNext,
    uint16 _baseRewardBps
) external restricted whenNotPaused(PauseCategory.ADMIN) {
    ...
    // Validate registered, no duplicates, and sum yield to BPS.
    uint256 total = _baseRewardBps;
    for (uint256 i; i < len; ++i) {
        address tranche = _tranches[i].tranche;
        @> if (!isTranche[tranche]) revert TrancheNotRegistered(tranche);
        ...
    }
    if (total != BPS) revert InvalidYieldDistribution();

    ...
}
```

```

        // Overwrite tranches array, validate pair contiguity and build pair
        ↪ mapping.
@>    tranches = _tranches;
        ...
    }

```

Problem is that it performs check only on registration. Suppose following example:

- 1) 40 tranches are registered
- 2) Admin calls `setTrancheConfig()` with single tranche
- 3) New 40 tranches are registered
- 4) As a result, admin can set config for 80 tranches, bypassing `MAX_TRANCHES = 50`.

Impact

It breaks invariant which is safe measure to avoid OOG error.

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/IssuanceVault.sol#L334>

Tool Used

Manual Review

Recommendation

There is no need in pushing to tranches at all, check length of new `_tranches`:

```

function registerTranche(
    address _tranche
) external restricted whenNotPaused(PauseCategory.ADMIN) {
    if (_tranche == address(0)) revert ZeroAddress();
    if (isTranche[_tranche]) revert TrancheExists();
-   if (tranches.length >= MAX_TRANCHES) revert TooManyTranches();

-   tranches.push(TrancheInfo(_tranche, 0, false));
    isTranche[_tranche] = true;
    emit TrancheRegistered(_tranche);
}

function setTrancheConfig(...) external restricted
    ↪ whenNotPaused(PauseCategory.ADMIN) {

```

```
uint256 len = _tranches.length;
if (len == 0) revert NoTranchesAdded();
+ if (len >= MAX_TRANCHES) revert TooManyTranches();
if (_isPairedWithNext.length != len) revert LengthMismatch();
...
}
```

Issue L-7: Incorrect condition in `Tranche.claim()`

Source: <https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/issues/29>

Summary

It doesn't charge fee if snapshot price is zero, however it is legitimate price.

Vulnerability Detail

There is a fee on positive yield during period between request creation and finalization. It skips fee if `snapshotPrice == 0`:

```
function claim(
    address user,
    uint256 epochId
) external whenDirectDisabled whenNotPaused(PauseCategory.WITHDRAWALS) {
    uint256 userShares = userSharesByEpoch[user][epochId];
    @> if (userShares == 0) revert NothingToClaim();
    ...
    uint256 snapshotPrice = userSnapshotPrice[user][epochId];
    if (
    @>     snapshotPrice != 0 &&
        unstakeYieldReductionBps > 0 &&
        finalizedData.finalizationPrice > snapshotPrice
    ) {
        uint256 yieldPerShare = finalizedData.finalizationPrice - snapshotPrice;
        uint256 slashAmount = (yieldPerShare * unstakeYieldReductionBps *
            ↪ userShares) /
            BASIS_POINTS /
            (10 ** decimals());
        if (slashAmount > userAssets) {
            slashAmount = userAssets;
        }
        userAssets -= slashAmount;
    }

    ...
}
```

However if it's 0, then it's legitimate price. If user has shares > 0 and `snapshotPrice == 0` then price was indeed 0 at request creation:

```
function requestWithdrawal(uint256 shares) external
    ↪ whenNotPaused(PauseCategory.WITHDRAWALS) {
    ...
}
```

```

        uint256 existingShares = userSharesByEpoch[msg.sender][targetEpoch];
        uint256 currentPrice = convertToAssets(10 ** decimals());
@>    if (existingShares == 0 || userSnapshotPrice[msg.sender][targetEpoch] == 0)
    ↪    {
@>        userSnapshotPrice[msg.sender][targetEpoch] = currentPrice;
        } else {...}
        ...
    }

```

So user will avoid fee if created request during 0 price. It is possible on risky Tranche to be fully seized.

Impact

User will avoid fee on edge case.

Code Snippet

<https://github.com/sherlock-audit/2026-02-mezzanine-feb-25th/blob/main/mezzanine-sc/contracts/Tranche.sol#L235>

Tool Used

Manual Review

Recommendation

Remove this condition:

```

        if (
-            snapshotPrice != 0 &&
            unstakeYieldReductionBps > 0 &&
            finalizedData.finalizationPrice > snapshotPrice
        ) {

```

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.